

Nafil Fadillah
20123958
3KB02

Pertemuan Minggu 4

```
In [1]: graph = {
'A' : ['Z', 'S', 'T'],
'Z' : ['O'],
'O' : ['S1'],
'S1' : ['F', 'R'],
'F' : ['B'],
'B' : [],
'R' : [],
'S' : ['O1', 'F1', 'R1'],
'O1' : [],
'F1' : ['B1'],
'B1' : [],
'R1' : ['C', 'P'],
'C' : [],
'P' : [],
'T' : ['L'],
'L' : ['M'],
'M' : ['D'],
'D' : []
}

In [2]: visited = [] # List for visited nodes.
queue = [] #Initialize a queue

In [3]: def bfs(visited, graph, node): #function for BFS
visited.append(node)
queue.append(node)

while queue:
    m = queue.pop(0)
    print (m, end = " ")

    for neighbour in graph[m]:
        if neighbour not in visited:
            visited.append(neighbour)
            queue.append(neighbour)

In [4]: # Driver Code
print("Following is the Breadth-First Search")
bfs(visited, graph, 'A')

Following is the Breadth-First Search
A Z S T O O1 F1 R1 L S1 B1 C P M F R D B

In [ ]:
```

Penjelasan Kode :

Kode yang diberikan merupakan implementasi algoritma **Breadth-First Search (BFS)** yang digunakan untuk menelusuri sebuah graf. Graf pada kode direpresentasikan dalam bentuk *dictionary* (adjacency list), di mana setiap node memiliki daftar node lain yang terhubung langsung dengannya. Misalnya, node A terhubung ke Z, S, dan T, yang berarti dari A kita bisa langsung menuju ketiga node tersebut.

Selanjutnya, terdapat dua variabel utama yaitu visited dan queue. Variabel visited berfungsi untuk menyimpan node-node yang sudah dikunjungi agar tidak dikunjungi kembali, sehingga menghindari perulangan tak terbatas. Sementara itu, queue digunakan sebagai antrian dalam proses BFS, yang bekerja dengan prinsip FIFO (*First In First Out*), yaitu data yang pertama masuk akan diproses lebih dulu.

Fungsi bfs dibuat untuk menjalankan proses penelusuran. Pada awalnya, node awal (dalam hal ini A) dimasukkan ke dalam visited dan juga ke dalam queue. Kemudian, program akan melakukan perulangan selama masih ada isi dalam

queue. Pada setiap iterasi, node paling depan dalam antrian akan diambil, lalu ditampilkan sebagai bagian dari hasil penelusuran.

Setelah itu, program akan mengecek semua node tetangga (neighbour) dari node yang sedang diproses. Jika ada tetangga yang belum pernah dikunjungi, maka node tersebut akan ditambahkan ke dalam visited dan juga dimasukkan ke dalam queue. Proses ini terus berulang hingga semua node yang dapat dijangkau telah dikunjungi.

Dengan cara kerja tersebut, BFS akan menelusuri graf secara melebar, yaitu mengunjungi semua node yang berada pada satu tingkat terlebih dahulu sebelum melanjutkan ke tingkat berikutnya. Hasil output yang ditampilkan merupakan urutan node yang dikunjungi selama proses BFS berlangsung, dimulai dari node A hingga seluruh node dalam graf selesai ditelusuri.